

GLA UNIVERSITY

Department of Computer Engineering & Applications

SOFTWARE REQUIREMENTS SPECIFICATION

BudgetSync

Smart Personal Finance Management System

A modern personal finance management platform designed to help users track expenses, manage budgets, monitor savings, and organize investments from a unified dashboard — with interactive analytics, visual spending insights, and financial reports for informed money-management decisions.

Prepared By	Krishna Tomar
Project Guide	Mr. Gautam Sir
University	GLA University
Document Version	1.0.0
Date	18 / 08 / 2026
Document Status	Final Draft



Table of Contents

Revision History	3
1. Introduction	4
1.1 Purpose	4
1.2 Document Conventions	4
1.3 Intended Audience	4
1.4 Project Scope	5
1.5 Definitions, Acronyms & Abbreviations	5
1.6 References	5
2. Overall Description	6
2.1 Product Perspective	6
2.2 Product Features Summary	6
2.3 User Classes and Characteristics	7
2.4 Operating Environment	7
2.5 Design & Implementation Constraints	8
2.6 Assumptions and Dependencies	8
3. System Features (Functional Requirements)	9
4. External Interface Requirements	15
5. System Architecture	17
6. Data Requirements & Database Design	19
7. Use Case Model	22
8. Non-Functional Requirements	24
9. Other Requirements	26
10. Appendix	27

Revision History

Version	Date	Description	Author
0.1.0	05/08/2026	Initial draft — scope and feature list	Krishna Tomar
0.5.0	12/08/2026	Added functional requirements & use cases	Krishna Tomar
1.0.0	18/08/2026	Final review, formatting, and approval draft	Krishna Tomar

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for **BudgetSync - Smart Personal Finance Management System**. It is written to provide a clear, complete, and unambiguous reference for the design, development, testing, and evaluation of the system. The document follows the general structure recommended by **IEEE Std 830** for Software Requirements Specifications, adapted to the scope of a college-level software engineering project.

This document is intended to serve as the single source of truth for what BudgetSync will do, who will use it, and the conditions under which it must operate, before implementation begins.

1.2 Document Conventions

Functional requirements are labeled using the format **FR-<Module>-<Number>** (e.g., **FR-TXN-01**). Non-functional requirements are labeled **NFR-<Number>**. Priority levels used throughout this document are:

Priority	Meaning
High	Core requirement; system is not usable without it (Phase 1 / MVP).
Medium	Important feature that adds significant value; targeted for Phase 2.
Low	Enhancement or stretch goal for future iterations.

1.3 Intended Audience

- **Project Guide / Faculty Evaluators** — to assess project scope, quality, and completeness.
- **Student Development Team** — as the primary technical blueprint for implementation.
- **UI/UX Designers** — to design wireframes and screens consistent with defined requirements.
- **Database / Backend Developers** — to design schema and APIs aligned with data requirements.
- **Software Testers** — to derive test cases directly from functional requirements.
- **End Users (indirect)** — individuals seeking a simple tool to manage personal finances.

1.4 Project Scope

BudgetSync is a web-based personal finance management application that allows an individual user to record income and expenses, define monthly budgets by category, track progress toward savings goals, maintain a simple record of personal investments, and view financial summaries through charts and reports. The system centers around a single-user model — each registered user manages their own private financial data.

In Scope: User registration/login, transaction tracking, category-wise budgeting, savings goal tracking, basic investment logging, dashboard analytics, downloadable/printable reports, and notification alerts for budget limits.
Out of Scope (for current academic version): Direct bank account integration / live transaction syncing, real-time stock market data feeds, multi-currency conversion, multi-user shared households, and payment gateway integration. These are identified as potential future enhancements in Section 10.2.

1.5 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
UI / UX	User Interface / User Experience
API	Application Programming Interface
MVP	Minimum Viable Product

CRUD	Create, Read, Update, Delete
JWT	JSON Web Token, used for secure session authentication
Budget Category	A user-defined or predefined grouping for expenses (e.g., Food, Rent, Travel)
Savings Goal	A target amount a user wishes to save by a specific date
Dashboard	The main landing screen summarizing a user's overall financial status

1.6 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- GLA University B.Tech Final/Minor Project Documentation Guidelines.
- Nielsen Norman Group — Usability Heuristics for User Interface Design.

2 Overall Description

2.1 Product Perspective

BudgetSync is a new, self-contained, standalone software product. It is not a modification of an existing system. It is designed as a three-tier web application consisting of a browser-based front-end client, a back-end application server exposing a REST API, and a relational database for persistent storage. The system is designed to be modular so that future enhancements (e.g., bank integration or mobile apps) can be added without redesigning the core architecture.

2.2 Product Features Summary

The table below summarizes the major functional modules covered in detail in Section 3.

#	Module	Description
1	User Authentication & Profile	Secure registration, login, and profile management.
2	Dashboard & Analytics	Unified visual summary of income, expenses, budgets, and savings.
3	Transaction Management	Add, edit, delete, and categorize income/expense transactions.
4	Budget Management	Create monthly/category budgets and track utilization in real time.
5	Savings Goals	Define and track progress toward personal savings targets.
6	Investment Tracking	Log and monitor a simple portfolio of personal investments.
7	Reports & Insights	Generate visual and exportable financial reports.
8	Notifications & Alerts	In-app alerts for budget overshoot and goal milestones.
9	Settings	Manage categories, currency preference, and account preferences.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Registered User	The primary user of the system who tracks personal income, expenses, budgets, and savings.	Low to moderate — general smartphone/computer literacy assumed.
Guest / Visitor	An unauthenticated visitor who can view the landing/marketing page but not access financial data.	Low
Administrator Phase 2	Manages system-level configuration such as default categories and monitors application health (future scope).	Moderate to high

2.4 Operating Environment

- **Client Side:** Any modern web browser (Chrome, Firefox, Edge, Safari) on Windows, macOS, Linux, Android, or iOS, with a minimum screen width of 360px (responsive design).
- **Server Side:** Node.js / Django (implementation team's choice) REST API server, deployable on cloud platforms such as Render, Railway, or a college server.
- **Database:** Relational database — MySQL or PostgreSQL.
- **Network:** Requires an active internet connection for all client-server communication.

2.5 Design and Implementation Constraints

- The project must be completed within a single academic semester by a small student team (typically 2–4 members).
- The system must use open-source / free-tier technologies only (no paid licenses), suitable for a college budget.
- All monetary values are stored and displayed in a single base currency (default: INR ₹) for the current version.
- The application must follow a RESTful API design for the client-server communication layer.
- The UI must strictly follow the defined color theme: Midnight Navy (#0F172A) and Emerald Green (#10B981) as primary brand colors.

2.6 Assumptions and Dependencies

- Users have basic familiarity with using web/mobile applications and entering financial data manually.
- Each user manages only their own personal data; no shared/family accounts are assumed in this version.
- The system assumes manual entry of transactions; no automatic bank feed is assumed to be available.
- Charting/visualization is assumed to be handled by a front-end charting library (e.g., Chart.js / Recharts).

3 System Features (Functional Requirements)

This section describes each functional module of BudgetSync, including its purpose, priority, and detailed functional requirements. Requirements are written to be specific, testable, and traceable.

3.1 User Authentication & Profile Management High Priority

Allows a new user to create a secure account and returning users to log in, ensuring that each user's financial data remains private and accessible only to them.

- FR-AUTH-01** — The system shall allow a new user to register using name, email address, and password.
- FR-AUTH-02** — The system shall validate that the email address is unique and properly formatted before account creation.
- FR-AUTH-03** — The system shall enforce a minimum password strength (minimum 8 characters, at least one number).
- FR-AUTH-04** — The system shall securely hash passwords before storing them in the database (e.g., bcrypt).
- FR-AUTH-05** — The system shall allow a registered user to log in using email and password and receive an authenticated session token (JWT).
- FR-AUTH-06** — The system shall allow a logged-in user to log out, invalidating the active session.
- FR-AUTH-07** — The system shall allow a user to update their profile information (name, currency preference, profile photo).
- FR-AUTH-08** — The system shall allow a user to reset a forgotten password via a registered email link. Medium

3.2 Dashboard & Financial Overview High Priority

Provides the user with a single unified view of their overall financial health upon login.

- FR-DASH-01** — The system shall display the user's total income, total expenses, and net balance for the current month on the dashboard.
- FR-DASH-02** — The system shall display a visual breakdown (pie/donut chart) of expenses by category for the selected period.
- FR-DASH-03** — The system shall display a trend chart (line/bar) of income vs. expenses over the last 6 months.
- FR-DASH-04** — The system shall display budget utilization summaries (e.g., "Food: 72% of ₹5,000 used") with a progress indicator.
- FR-DASH-05** — The system shall display a summary of active savings goals and their completion percentage.
- FR-DASH-06** — The system shall allow the user to filter dashboard data by a selected month or custom date range.

3.3 Transaction Management High Priority

Enables users to record and manage all income and expense entries, forming the core data of the system.

- FR-TXN-01** — The system shall allow a user to add a transaction with amount, type (income/expense), category, date, and optional note.

FR-TXN-02 — The system shall allow a user to edit an existing transaction.

FR-TXN-03 — The system shall allow a user to delete a transaction, with a confirmation prompt.

FR-TXN-04 — The system shall display a list of all transactions, sortable by date, amount, or category.

FR-TXN-05 — The system shall allow a user to search and filter transactions by category, type, date range, or keyword.

FR-TXN-06 — The system shall allow a user to mark a transaction as recurring (e.g., monthly rent). **Medium**

FR-TXN-07 — The system shall validate that the transaction amount is a positive numeric value.

3.4 Budget Management **High Priority**

Allows users to plan and monitor spending limits for each expense category on a monthly basis.

FR-BUD-01 — The system shall allow a user to set a monthly budget limit for each expense category.

FR-BUD-02 — The system shall automatically calculate the amount spent against each budget as related transactions are added.

FR-BUD-03 — The system shall visually indicate budget status using color coding (Emerald Green = within budget, Amber = nearing limit, Red = exceeded).

FR-BUD-04 — The system shall allow a user to edit or delete an existing budget.

FR-BUD-05 — The system shall allow a user to copy the previous month's budget plan to the current month. **Medium**

3.5 Savings Goals **High Priority**

Enables users to define specific savings targets and track progress toward achieving them.

FR-SAV-01 — The system shall allow a user to create a savings goal with a name, target amount, and target date.

FR-SAV-02 — The system shall allow a user to log manual contributions toward a savings goal.

FR-SAV-03 — The system shall display progress toward each goal as a percentage and progress bar.

FR-SAV-04 — The system shall notify the user when a savings goal is fully achieved.

FR-SAV-05 — The system shall allow a user to edit or close/delete a savings goal.

3.6 Investment Tracking **Medium Priority**

Provides a simplified way for users to record and view their personal investments without requiring live market data integration.

FR-INV-01 — The system shall allow a user to manually add an investment entry with type (e.g., FD, Mutual Fund, Stocks, Gold), invested amount, and date.

FR-INV-02 — The system shall allow a user to manually update the current value of an investment.

FR-INV-03 — The system shall calculate and display the overall gain/loss (absolute and percentage) per investment.

FR-INV-04 — The system shall display total portfolio value and total invested amount in a summary card.

3.7 Reports & Insights High Priority

Allows users to generate summarized views of their financial activity over selected periods for review and record-keeping.

FR-REP-01 — The system shall generate a monthly financial summary report showing total income, expenses, savings, and net balance.

FR-REP-02 — The system shall allow a user to export a report as a PDF or CSV file.

FR-REP-03 — The system shall display a category-wise expense comparison across multiple months.

FR-REP-04 — The system shall highlight the top 3 spending categories for the selected period.

3.8 Notifications & Alerts Medium Priority

FR-NOT-01 — The system shall display an in-app alert when spending in a category crosses 80% of its budget.

FR-NOT-02 — The system shall display an in-app alert when a budget category exceeds 100% of its limit.

FR-NOT-03 — The system shall display a congratulatory alert when a savings goal is completed.

3.9 Settings & Preferences Medium Priority

FR-SET-01 — The system shall allow a user to add, edit, or remove custom expense/income categories.

FR-SET-02 — The system shall allow a user to select a preferred currency symbol for display.

FR-SET-03 — The system shall allow a user to switch between light and dark display themes. Low

FR-SET-04 — The system shall allow a user to permanently delete their account and all associated data.

4 External Interface Requirements

4.1 User Interfaces

- A responsive web interface accessible via desktop and mobile browsers, built following the BudgetSync visual theme (Midnight Navy & Emerald Green).
- Primary screens: Login/Register, Dashboard, Transactions, Budgets, Savings Goals, Investments, Reports, Settings.
- Navigation shall be provided through a persistent sidebar (desktop) / bottom navigation bar (mobile).
- All forms shall provide inline validation feedback (e.g., red outline with message for invalid input).
- Charts and key positive indicators (income, savings progress, within-budget status) shall use Emerald Green; warnings shall use amber/red for clear contrast.

4.2 Hardware Interfaces

The system does not require any specialized hardware. It requires only a standard computing device (desktop, laptop, tablet, or smartphone) with a functioning display, input device, and internet connectivity.

4.3 Software Interfaces

Interface	Purpose
Web Browser (Client)	Renders the front-end single-page application (SPA).
REST API (Backend)	Exposes JSON-based endpoints for authentication, transactions, budgets, goals, investments, and reports.
Relational Database (MySQL/PostgreSQL)	Persists all user and financial data.
Charting Library (e.g., Chart.js/Recharts)	Renders dashboard and report visualizations on the client.
Email Service (SMTP/3rd-party API)	Sends password reset and alert emails. Medium

4.4 Communication Interfaces

- All client-server communication shall occur over HTTPS.
- Data exchange format between client and server shall be JSON.
- Authenticated requests shall include a Bearer token (JWT) in the request header.

5 System Architecture

BudgetSync follows a standard three-tier architecture, separating the presentation layer, business logic layer, and data layer. This separation allows the front-end and back-end to be developed and tested independently by different team members — a practical structure for a student project team.

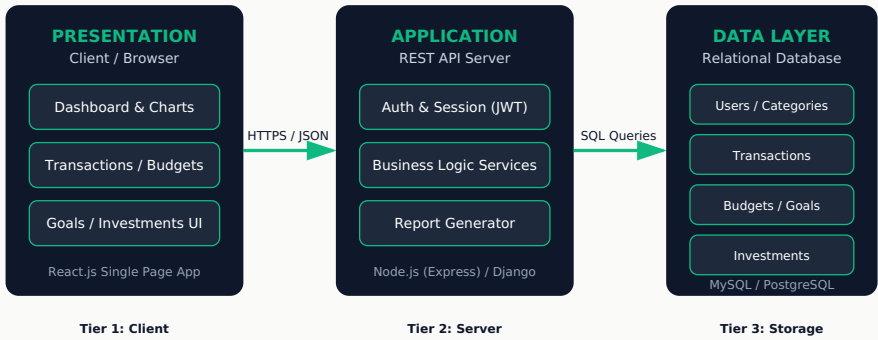


Figure 5.1 — High-Level Three-Tier System Architecture of BudgetSync

5.1 Architecture Layers

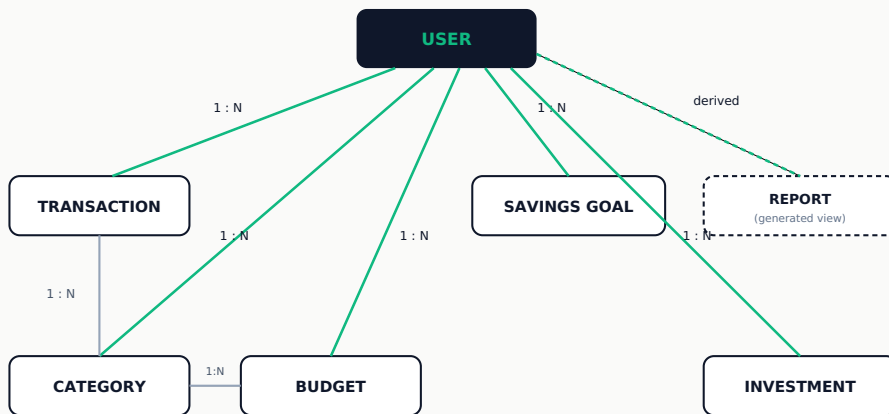
Layer	Responsibility	Suggested Technology
Presentation Layer	Renders UI, handles user interaction, calls REST APIs.	React.js / HTML-CSS-JS
Application Layer	Business logic — authentication, budget calculations, report generation.	Node.js (Express) / Django REST Framework
Data Layer	Stores and retrieves persistent data (users, transactions, budgets, goals, investments).	MySQL / PostgreSQL

5.2 Module Interaction Overview

The front-end communicates exclusively through the REST API layer; it never accesses the database directly. The application layer enforces authentication (via JWT), applies business rules (e.g., budget threshold checks), and returns structured JSON responses that the presentation layer renders as charts, tables, and cards.

6 Data Requirements & Database Design

This section outlines the core data entities required to support BudgetSync's functionality, intended to guide database schema design in the implementation phase.



A single USER owns all Transactions, Categories, Budgets, Savings Goals, and Investments (1 : N relationships)

Figure 6.1 — Simplified Entity Relationship Overview

6.1 Core Entities

User

Field	Type	Description
user_id (PK)	INT	Unique identifier
name	VARCHAR	Full name of user
email	VARCHAR (unique)	Login email address
password_hash	VARCHAR	Securely hashed password
currency_pref	VARCHAR	Preferred currency symbol (default ₹)
created_at	DATETIME	Account creation timestamp

Transaction

Field	Type	Description
txn_id (PK)	INT	Unique identifier
user_id (FK)	INT	Owner of the transaction
category_id (FK)	INT	Associated category
type	ENUM	'income' or 'expense'
amount	DECIMAL(10,2)	Transaction amount
txn_date	DATE	Date of transaction
note	VARCHAR	Optional description
is_recurring	BOOLEAN	Flag for recurring transactions

Category

Field	Type	Description

category_id (PK)	INT	Unique identifier
user_id (FK)	INT	Owner (null for default/system categories)
name	VARCHAR	Category name (e.g., Food, Rent)
type	ENUM	'income' or 'expense'

Budget

Field	Type	Description
budget_id (PK)	INT	Unique identifier
user_id (FK)	INT	Owner of the budget
category_id (FK)	INT	Category being budgeted
month_year	DATE	Applicable month (e.g., 2026-08-01)
limit_amount	DECIMAL(10,2)	Budget ceiling for the category

SavingsGoal

Field	Type	Description
goal_id (PK)	INT	Unique identifier
user_id (FK)	INT	Owner of the goal
name	VARCHAR	Goal title (e.g., "New Laptop")
target_amount	DECIMAL(10,2)	Target savings amount
current_amount	DECIMAL(10,2)	Amount saved so far
target_date	DATE	Target completion date

Investment

Field	Type	Description
investment_id (PK)	INT	Unique identifier
user_id (FK)	INT	Owner of the investment
type	VARCHAR	Investment type (FD, Stocks, Mutual Fund, Gold, etc.)
invested_amount	DECIMAL(10,2)	Amount originally invested
current_value	DECIMAL(10,2)	Latest known value
invested_date	DATE	Date of investment

6.2 Data Integrity Rules

- Every transaction, budget, goal, and investment record must be linked to exactly one valid user_id.
- Deleting a user account shall cascade-delete all associated financial records (per FR-SET-04).
- Amount fields shall not accept negative or non-numeric values at the API validation layer.

7 Use Case Model

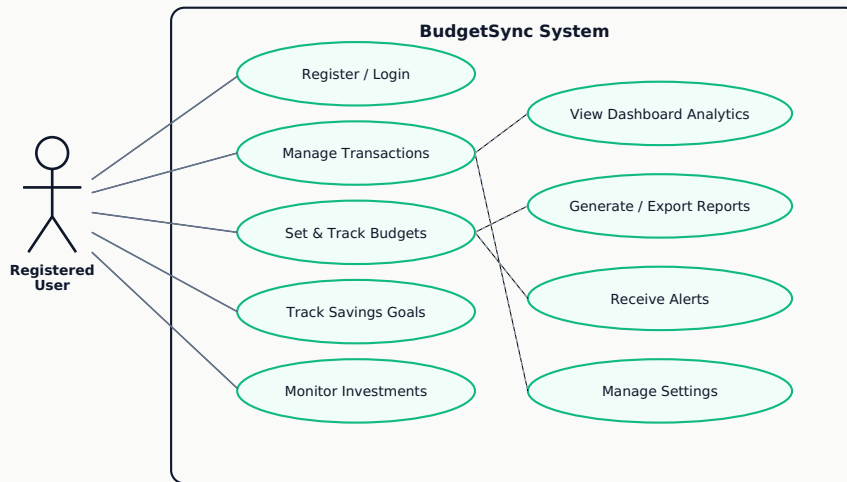


Figure 7.1 — Use Case Diagram for BudgetSync (Registered User)

7.1 Primary Use Case — Add a Transaction

Use Case ID	UC-01
Actor	Registered User
Description	User records a new income or expense transaction.
Preconditions	User is logged in.
Main Flow	1. User navigates to Transactions screen and selects "Add Transaction". 2. User enters amount, selects type (income/expense), category, and date. 3. User submits the form. 4. System validates input and saves the transaction. 5. System updates dashboard totals and related budget usage.
Alternate Flow	If validation fails (e.g., negative amount), system displays an inline error and the transaction is not saved.
Postcondition	Transaction is stored; dashboard and budget views reflect updated totals.

7.2 Primary Use Case — Set a Monthly Budget

Use Case ID	UC-02
Actor	Registered User
Description	User defines a spending limit for a category for the current month.
Preconditions	User is logged in; at least one category exists.
Main Flow	1. User navigates to Budgets screen. 2. User selects a category and enters a limit amount for the month. 3. System saves the budget and begins tracking related expense transactions. 4. System displays real-time usage percentage as expenses are logged.
Postcondition	Budget is active for the selected month and category.

7.3 Use Case Summary Table

ID	Use Case	Actor	Related Requirements
UC-01	Add / Edit / Delete Transaction	Registered User	FR-TXN-01 to FR-TXN-07
UC-02	Set / Track Monthly Budget	Registered User	FR-BUD-01 to FR-BUD-05
UC-03	Create & Track Savings Goal	Registered User	FR-SAV-01 to FR-SAV-05
UC-04	Log & Monitor Investment	Registered User	FR-INV-01 to FR-INV-04
UC-05	View Dashboard Analytics	Registered User	FR-DASH-01 to FR-DASH-06
UC-06	Generate / Export Report	Registered User	FR-REP-01 to FR-REP-04
UC-07	Register / Login / Logout	Registered User, Guest	FR-AUTH-01 to FR-AUTH-08

8 Non-Functional Requirements

8.1 Performance Requirements

NFR-01 — The dashboard shall load within 3 seconds under normal network conditions (for a dataset of up to 5,000 transactions).

NFR-02 — API responses for standard CRUD operations shall return within 500ms on average.

8.2 Security Requirements

NFR-03 — All passwords shall be stored using a strong one-way hashing algorithm (e.g., bcrypt); plaintext passwords shall never be stored.

NFR-04 — All API endpoints handling personal financial data shall require a valid authentication token.

NFR-05 — All data transmission between client and server shall be encrypted using HTTPS/TLS.

NFR-06 — A user shall only be able to access, modify, or delete their own financial records.

8.3 Usability Requirements

NFR-07 — The interface shall be intuitive enough for a first-time user to add a transaction within 3 clicks/taps from the dashboard.

NFR-08 — The UI shall maintain consistent visual language (Midnight Navy & Emerald Green theme) across all screens.

NFR-09 — The application shall be fully responsive and usable on screen sizes from 360px (mobile) to 1920px (desktop).

8.4 Reliability & Availability

NFR-10 — The system shall correctly persist all committed transactions even if the client disconnects immediately after submission.

NFR-11 — The system shall target 99% uptime during the academic evaluation and demonstration period.

8.5 Maintainability & Scalability

NFR-12 — The codebase shall follow a modular structure (separate modules for auth, transactions, budgets, goals, investments) to support future feature additions.

NFR-13 — The database schema shall be normalized to at least 3NF to minimize data redundancy.

8.6 Portability

NFR-14 — The application shall function correctly on the latest two major versions of Chrome, Firefox, and Edge browsers.

9 Other Requirements

9.1 Legal & Compliance

As BudgetSync handles sensitive personal financial information (even if manually entered and not linked to real bank accounts), the system shall follow basic data protection best practices: data shall not be shared with third parties, and users shall be able to permanently delete their account and data on request (FR-SET-04).

9.2 Future Enhancements (Post-Academic Scope)

- Integration with bank/UPI APIs for automatic transaction import.
- Real-time stock/mutual fund price feeds for live investment valuation.
- Multi-currency support with live exchange rate conversion.
- Shared/family budgeting with multiple linked user accounts.
- Native mobile application (Android/iOS) using the same backend API.
- AI-based spending pattern insights and personalized saving recommendations.

9.3 Testing Considerations

Each functional requirement in Section 3 is designed to map directly to one or more test cases during the testing phase. Suggested testing types include:

Test Type	Focus Area
Unit Testing	Individual functions — e.g., budget percentage calculation, password hashing.
Integration Testing	API endpoints interacting correctly with the database (e.g., adding a transaction updates budget usage).
UI / Usability Testing	Navigation flow, responsiveness, and adherence to the visual theme.
Security Testing	Authentication enforcement, data isolation between users.
User Acceptance Testing (UAT)	Faculty/peer review against functional requirements checklist.

10 Appendix

10.1 Visual Theme Reference

Element	Color	Hex Code	Usage
Primary	Midnight Navy	#0F172A	Headings, navigation, structural elements
Accent	Emerald Green	#10B981	Highlights, positive indicators, charts, CTAs
Background	Off-White	#FAFAF9	Page and screen backgrounds
Body Text	Dark Charcoal	#2A2E35	Primary readable text

10.2 Project Team & Approval

Prepared By	Krishna Tomar
Project Guide	Mr. Gautam Sir
Institution	GLA University
Document Version	1.0.0
Date of Issue	18/08/2026

End of Document. This SRS represents the agreed scope for BudgetSync version 1.0 and shall serve as the baseline reference for design, development, and evaluation of the project. Any change in scope shall be documented as a revision in the Revision History table.